

Normalização:

Análise de qualidade:

Critérios informais – clareza da semântica dos atributos da relação, redundância de informação no tuplo, redução dos NULLS nos tuplos, junção de relações baseada em PK e FK

Critérios formais

Processo de normalização:

Formas normais, baseadas em critérios formais

Dependências funcionais:

$X \rightarrow Y$ – Y é funcionalmente dependente de X, os valores da componente X do tuplo define de forma única a componente Y do respetivo tuplo.

Dependência parcial: atributo depende de parte dos atributos que compõem a chave da relação

Dependência transitiva: atributo que não faz parte da chave da relação depende de um atributo que também não faz parte da relação

Dependência total: atributo depende de toda a chave da relação

Cada forma superior tem menos DF que a anterior.

Relações que não satisfazem os testes de determinada forma normal são decompostas em relações menores.

Primeira forma normal:

Atributos são atômicos (não permite atributos composto ou multivalor)

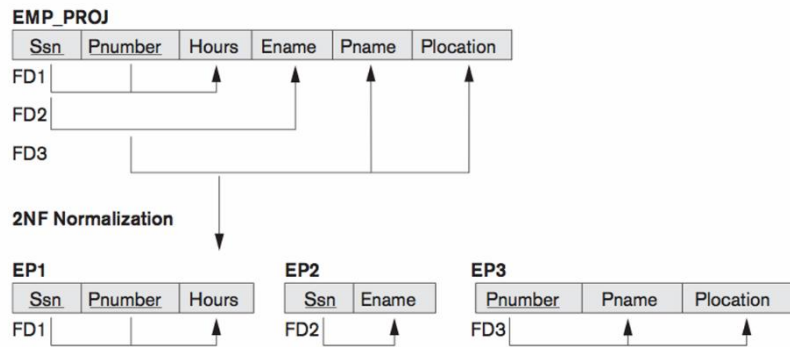
Não suporta relações dentro de relações

Para resolver separar em tabelas distintas

Segunda forma normal:

Está na 1FN

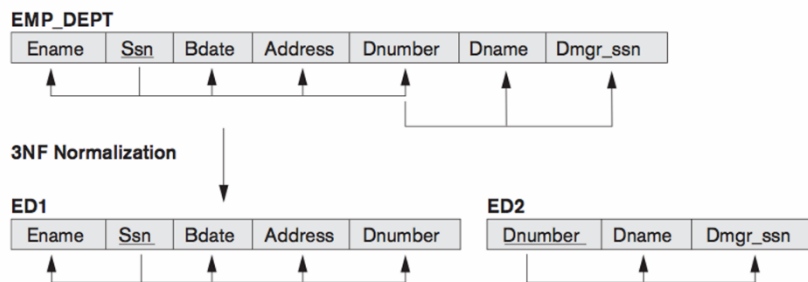
Não podem existir dependências parciais (todos os atributos não pertencentes a qualquer chave candidata devem depender totalmente da chave e não de parte dela)



Terceira forma normal:

Está na 2FN

Não podem existir dependências transitivas (não existem dependências funcionais entre atributos não chave)



BCNF:

Definição: todos os atributos são funcionalmente dependentes da chave da relação, de toda a chave e de nada mais

4FN:

Está na BCNF

Não existem dependências multivalor

5FN:

Está na 4FN

A relação não pode ser mais decomposta sem haver perda de informação

Não existem dependências de junção

Índices:

São estruturas de dados que oferecem uma segunda forma (rápida) de acesso aos dados.

- Melhora o tempo de consulta – crítico para desempenho da BD

- Pode aumentar o volume de dados armazenado (overhead) e o tempo das inserções

Num SGBD os tuplos de uma relação estão distribuídos (armazenados) por várias páginas (blocos) em disco

Os índices são estruturas que:

- Têm um valor ordenado (atributo indexado)

- Um ponteiro para a sua localização

 - Não denso: Início da página (Bloco)

 - Denso: Offset do próprio tuplo na página

Single-level ordered

Single-level index:

- Primary index

- Clustered index

- Secondary index

Multilevel index:

- Log fo bi – fo = fan out e fo > 2

Estes índices são tipicamente implementados com estruturas em árvores balanceadas (equilibradas): B-Tree

Uma árvore de pesquisa é balanceada se a distância de qualquer folha ao nó raiz for sempre a mesma

SQL server tem dois tipos de índices (clustered – os nós folha contêm os próprios dados da relação; non-clustered – os índices apontam para a tabela base que pode ser clustered ou heap)

Page split quando a página está cheia para uma inserção – penalizador em termos de desempenho temporal

Opções de especialização:

Unique:

Index key é única

Por defeito, a criação de uma chave primária cria automaticamente um unique clustered index

Composite:

Índice com vários atributos

A ordem dos atributos importa

Filtered:

Permite utilização da cláusula WHERE no CREATE INDEX

Só disponível para non-clustered index

Só indexamos parte dos tuplos da relação

Inclusão de atributos num Índice:

Podemos incluir non-key atributos nas folhas de um índice non-clustered

Chamadas query “cobertas”

B-Tree Tuning – minimizar os page splits

Fill factor – permite definir a % de espaço livre

Pad index – indica se só aplicamos o fill factor aos nós folhas (ou não)

Utilizar fill factor próximo de 100% se temos inserções ordenadas

65-85% se tivermos mais inserções no meio da B-Tree

SQL Programming

Batch: Grupo de uma ou mais instruções SQL que constituem uma unidade lógica.

Um erro sintático numa instrução provoca a falha de toda a batch.

Um erro de runtime não anula instruções SQL prévias (nessa batch).

São delimitadas pelo terminador “GO”

GO não é enviado para o servidor

“GO n” – executa a batch n vezes

Terminada a batch são eliminadas todas as variáveis locais, tabelas temporárias e cursores criados.

Script: Trata-se de um ficheiro de texto contendo uma ou mais batches delimitadas por GO. As batch são executadas em sequência.

Tabelas temporárias locais:

São sinalizadas com o carácter # antes do nome

São criadas na base de dados tempdb

Estão visíveis:

Só na sessão que as criou

No level em que são criados e todos os inner level (da call stack)

São eliminadas quando o procedimento ou função terminam

Tabelas temporárias globais:

Utilizamos dois ## antes do nome

Similares às local temporary tables (tempdb) mas têm um scope maior

Ficam visíveis para outras sessões – apropriadas para partilha de dados, todos têm full access

São eliminadas quando a última sessão desconecta

Tabelas como variáveis:

São similares a tabelas temporárias locais mas têm um scope mais limitado:

Tem o mesmo scope que as variáveis locais

Mas não estão visíveis em inner levels da call stack

Declaram-se como variáveis – também tem existência na tempdb

Desaparecem quando a batch, procedimento ou função, onde foram criadas, chega ao fim

Limitadas em termos de restrições:

Não é permitido: chaves estrangeiras e check

Permitido: chaves primária, defaults, nulls, unique

Não podem ter objetos dependentes

Chaves estrangeiras ou triggers

Cursor: Ferramenta que permite percorrer sequencialmente os tuplos retornados por determinada consulta (SELECT)

Set based query vs cursor operation – set-based geralmente são mais rápidas

Quando cursors são uma melhor opção?

Iterar uma stored procedure, iterar código DDL, somas cumulativas, dados time-sensitive

O cursor pode ser static, keyset, dynamic, fast-forward

Stored procedures:

Batch armazenada com um nome. Os procedimentos são guardados na memória cache na primeira vez em que são executados.

SQL Statement

First Time

- Check syntax
- Compile
- Execute
- Return data

Second Time

- Check syntax
- Compile
- Execute
- Return data

⋮

Stored Procedure

Creating

- Check syntax
- Compile

First Time

- Execute
- Return data

Second Time

- Execute
- Return data

⋮

System stored procedure

- Criados na Master database

- Podem ser utilizados em qualquer base de dados

Local stored procedure

- São definidos num base de dados local

- Nome livre mas recomenda-se uma normalização por parte do utilizador

T-SQL Error Handling

@@error: retorna um inteiro com o código de erro da última instrução. 0 – Sucesso

@@rowcount: permite saber quantos tuplos foram afectados por determinada instrução SQL

T-SQL RAISERROR – retorna uma mensagem de erro ao cliente

UDF:

Podem ser utilizados para incorporar lógica complexa dentro de uma consulta

Oferecem os mesmos benefícios das vistas pois podem ser utilizados como fonte de dados nas consultas e nas cláusulas WHERE/HAVING. Aceita parâmetros ao contrário das views.

3 tipos de UDF:

- Escalares

- Aceitam múltiplos parametros, retornam um único valor, podem ser utilizados dentro de qualquer expressão T-SQL, incluindo check constraint

- Determinísticas – o mesmo valor de entrada produz o mesmo retorno

- Inline table-valued

- Similar a vistas mas pode ter parametros de entrada

- Multi-statement table-valued functions

- Combina a capacidade das funcoes escalares (conter código complexo) com a capacidade das inline table-valued (retornar um conjunto).

SP versus UDF

- | | |
|---|--|
| • return - zero, single or multiple values | • return - single value (scalar or table) |
| • input/output param | • input param |
| • cannot use SELECT/ WHERE/ HAVING statement | • can use SELECT/ WHERE/ HAVING statement |
| • call SP - OK | • call SP - NOK |
| • exception handling - OK | • exception handling - NOK |
| • transactions - OK | • transaction - NOK |

Trigger: um tipo especial de stored procedure que é executado em determinadas circunstâncias (eventos) associadas à manipulação de dados.

SQL server suporta dois tipos de trigger: DML e DDL. Só vamos tratar de DML.

SQL server triggers são disparados uma vez por cada operação de modificação de dados.

INSTEAD OF: a transação para no ponto 4, não são recursivos, é ignorado na segunda vez, pode contornar problemas de integridade mas não de nulidade, tipo de dados e identidade das colunas, usar quando sabemos que a probabilidade de rollback é grande

AFTER: no fim apenas antes do commit podendo fazer rollback da transação se os dados forem inaceitáveis, pode assumir que os dados passaram todos as verificações de integridade dos dados, ocorre depois de todos os constraints.

Transações

Conjunto de operações de leitura e escrita sobre a base de dados

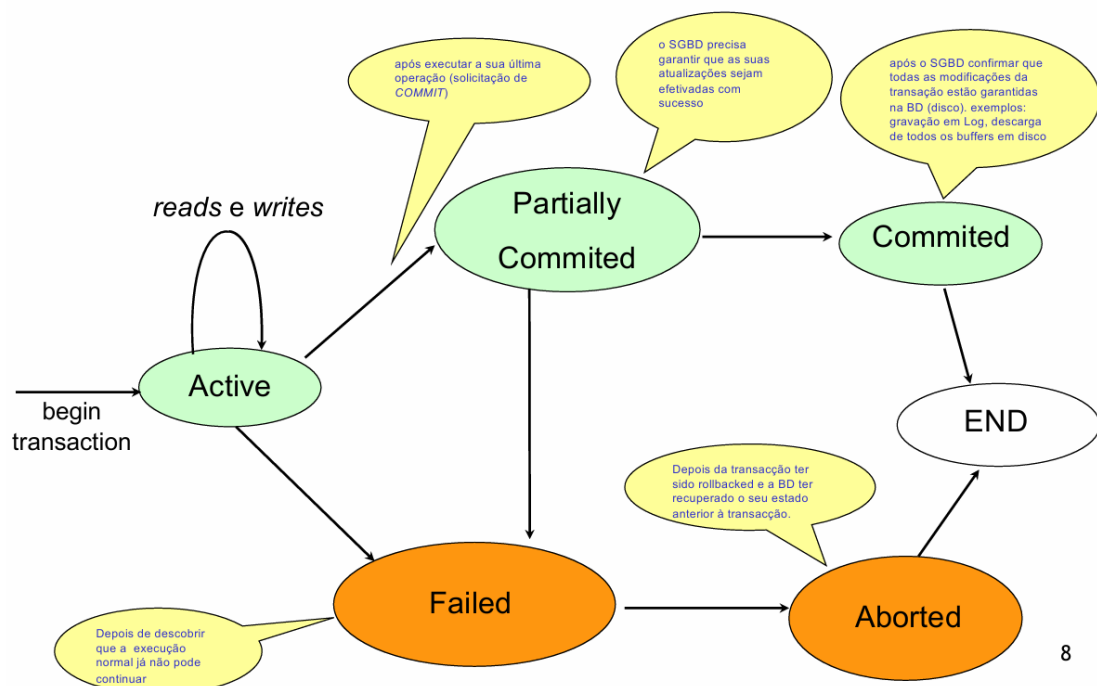
Read(x)- elemento x da base de dados para a memória volátil

Write(x) – elemento x da memória para a base de dados

SET TRANSACTION – inicia e configura características de uma transacao

COMMIT – encerra a transacao (solicita efetivação das ações)

ROLLBACK – solicita que as ações da transação sejam desfeitas



8

Controlo da concorrência

Escalonamento serializado – transação executada de cada vez, ineficiente

Escalonamento concorrente serializado – execução concorrente de transações mas de modo a preservar o isolamento, mais eficiente (podem existir sequências distintas com resultados distintos)

Um determinado escalonamento determina a ordem das transações

Problemas:

Atualização perdida (lost-update) – uma transação T1 grava um dado que entretanto já tinha sido lido e utilizado na transação T2

T1	T2
read(A)	
A = A - 40	
	read(A)
	A = A + 10
write(A)	
read(B)	
	write(A)
B = B + 20	
write(B)	

**A atualização de A
efetuada por T1 foi
perdida!**

Leitura suja (dirty-read) – T1 atualiza um elemento A e, posteriormente, outras transações leem A. No entanto T1 falha e as suas operações são desfeitas

T1	T2
read(A)	
A = A - 10	
write(A)	
	read(A)
	A = A + 20
	write(A)
read(Y)	
abort()	

**A transação T1
falha e deve
repor o valor
que A tinha
antes de T1
iniciar.**

**T2 leu um valor
de A que mais
tarde será
rollbacked!**

20

Métodos de controlo da concorrência:

Mecanismos de locking – preventivo

Locks binários – só permitem acessos exclusivos

Lock leitura/escrita - Apenas os acessos para escrita são exclusivos

Libertados no fim da transação 8commit ou rollback

Mecanismos de etiquetagem - preventivo

Métodos optimistas – optimista

Mecanismos de recuperação:

Escalonamentos

Backups

Transaction logging

Falha de disco – after images

Falha de transação – before image

Falha de sistema – before image e after image

NoSQL

Desfasamento de impedância – fragmentação das tabelas relacionais

Atributos chave incluem:

- Não relacional

- API simples

- BASE & Teorema CAP

- Sem esquema

- Inerentemente distribuído

- Open Source

ACID – é como um cofre bancário

BASE – é como uma conversa numa rede social

Teorema CAP de Brewer

Apenas pode ter duas das seguintes características:

- Consistência

- Disponibilidade

- Tolerância a partições

Designers são forçados a decidir entre consistência e disponibilidade

- CP — Consistência/Tolerância a Partições: quando ocorre uma partição de rede, o sistema recusa pedidos para manter a consistência (devolve erro).
- AP — Disponibilidade/Tolerância a Partições: quando ocorre uma partição de rede, o sistema continua a responder a pedidos, mas pode devolver dados desatualizados.

Tipos principais

- Armazéns chave-valor - REDIS

- Armazéns de documentos - mongoDB

Armazéns de colunas – Apache Cassandra

Bases de dados de grafos – Neo4j

Bases de dados de séries temporais - InfluxDB

Tipos não principais:

- Bases de dados orientadas a objetos

- Bases de dados XML nativas

- Armazéns RDF